

Proyecto 7: Monitoreo de Infraestructura con Zabbix 6.x sobre Docker

Juan Camilo Ballesteros Sierra · Luis Felipe Murillo Matallana

Juan Sebastián Delgado · Daniela Castro Quiñones

Servicios Telemáticos, Universidad Autónoma de Occidente, Semestre 2026-01

Repositorio: <https://github.com/ballesterossmartsolutionssas/Proyecto7-Zabbix>

Abstract—Este artículo presenta el diseño, implementación y validación de una plataforma de monitoreo de infraestructura basada en Zabbix 6.x, Docker y Docker Compose. La solución observa una red de servicios contenedorizados compuesta por una aplicación web, una base de datos MariaDB, un servicio DNS CoreDNS y un servicio FTP VSFTPD. Cada host cuenta con agente Zabbix, monitoreo de disponibilidad, métricas de CPU, memoria y disco, triggers de falla y alertas SMTP. Como valor agregado, el servicio web monitoreado se extendió a una aplicación real con frontend, backend Node.js, persistencia en MariaDB, endpoints JSON, exporter /metrics, panel de analíticas, SLO de laboratorio y escenarios de carga con Artillery. La solución fue publicada en una VPS con HTTPS y subdominios públicos, lo que permitió sustentar pruebas en vivo de carga, caída controlada, alertas en MailHog, recuperación y métricas históricas. Los resultados muestran cumplimiento completo de los requerimientos del Proyecto 7 y una ampliación operativa hacia observabilidad aplicada.

Keywords—Zabbix 6.x, Docker Compose, monitoreo, observabilidad, MailHog, Artillery, SLO, infraestructura.

I. INTRODUCCIÓN

Las infraestructuras telemáticas modernas dependen de varios componentes que deben mantenerse disponibles de forma simultánea. Una aplicación web puede responder correctamente, pero si la base de datos, DNS o FTP fallan, el servicio completo se degrada. Por esta razón, el monitoreo centralizado no debe limitarse a revisar si un servidor está encendido, sino que debe medir disponibilidad, recursos, servicios, eventos y comportamiento bajo carga.

El Proyecto 7 planteó implementar una plataforma de monitoreo usando Zabbix 6.x, agentes, base de datos y Docker Compose. A partir de ese enunciado se construyó una solución reproducible con cuatro servicios monitoreados, frontend web de Zabbix, templates, triggers y alertas por correo. Además, se agregó una capa de valor para la sustentación: una consola pública que genera tráfico, escribe telemetría, consulta MariaDB, muestra SLO, expone métricas Prometheus y facilita pruebas con Artillery.

El objetivo del trabajo fue demostrar el ciclo completo de operación: desplegar infraestructura, monitorearla, generar una falla, visualizar el problema en Zabbix, recibir la alerta, recuperar el servicio y analizar la evidencia histórica. El enfoque busca acercar el laboratorio a un escenario real de operación, donde la observabilidad permite tomar decisiones antes de que el usuario final reporte una interrupción.

La contribución principal del proyecto es unir el monitoreo clásico de infraestructura con una demostración funcional. En lugar de presentar un portal estático, el servicio web genera solicitudes, registra incidentes, produce métricas y permite observar cambios durante la exposición. Esto facilita explicar al evaluador cómo se comporta la plataforma cuando hay tráfico normal, carga sostenida, caída de servicios y recuperación.

II. CONTEXTO DEL PROBLEMA

En entornos basados en contenedores, cada servicio se ejecuta de manera aislada y puede fallar sin afectar inmediatamente a los demás. Esta característica mejora la modularidad, pero también dificulta la visibilidad si no existe un sistema de monitoreo. La detección manual es lenta, no conserva historia y no permite diferenciar entre

caída total, degradación por carga o error puntual de una dependencia.

El problema central consiste en observar una red heterogénea de servicios: HTTP, base de datos, DNS y FTP. La plataforma debía responder preguntas operativas concretas: si el host está disponible, si el puerto del servicio responde, si los recursos se comportan dentro de rangos esperados, si la caída genera un evento visible, si el correo de alerta llega y si las métricas permiten explicar lo ocurrido después de la recuperación.

La rúbrica del curso también exige estructurar el proceso de validación y construir un prototipo funcional. Por ello, se incorporaron pruebas automatizadas, auditoría de cumplimiento, evidencias exportables y una aplicación web capaz de producir carga real. Esta ampliación evita que el proyecto sea únicamente una instalación de Zabbix y lo convierte en una demostración verificable de monitoreo de infraestructura.

Desde la perspectiva de operación, el riesgo no está solo en que un contenedor se detenga. También puede ocurrir que el contenedor esté arriba pero el servicio no responda, que la base de datos no tenga datos, que el endpoint público falle por HTTPS, o que la carga incremente los tiempos de respuesta. La solución debía cubrir estos casos por medio de agent checks, checks TCP, web escenarios y endpoints propios de la aplicación.

III. ALTERNATIVAS DE SOLUCIÓN

Se evaluaron Nagios Core, Prometheus con Grafana, Datadog y Zabbix 6.x. Los criterios fueron costo, despliegue local, soporte para Docker, facilidad de aprovisionamiento, alertas, dashboards y adecuación al tiempo disponible. La comparación no buscó escoger la herramienta más poderosa en términos absolutos, sino la más coherente con el alcance académico y con la necesidad de entregar un entorno reproducible en Docker Compose.

Tabla I. Comparación de alternativas de monitoreo.

Nagios Core: Checks maduros y ecosistema amplio, pero interfaz limitada y menor automatización moderna. No seleccionada.

Prometheus + Grafana: Métricas modernas, PromQL y visualización fuerte, pero requiere Alertmanager y exporters adicionales.

Datadog: Suite SaaS completa con APM, logs y ML; descartada por costo, dependencia externa y datos fuera del laboratorio.

Zabbix 6.x: Agentes, templates, API, dashboards, histórico y alertas integradas. Alternativa seleccionada.

A. Análisis de alternativas

Nagios Core fue considerado por su madurez y por la disponibilidad de plugins para verificar puertos y servicios. Sin embargo, su configuración se apoya fuertemente en archivos y su interfaz resulta menos adecuada para mostrar dashboards y evidencias durante una sustentación. Para el proyecto, la capacidad de aprovisionar con API y mostrar métricas históricas de forma integrada era más importante que la cantidad de plugins disponibles.

Prometheus con Grafana es una alternativa fuerte para ambientes cloud-native. Su modelo de series temporales y etiquetas es muy flexible, y Grafana ofrece dashboards superiores visualmente. Aun así, para cubrir los mismos servicios del proyecto habría sido necesario integrar varios exporters, Alertmanager y configuración adicional. Esa arquitectura es poderosa, pero introduce más piezas que explicar, desplegar y validar dentro del tiempo del curso.

Datadog fue evaluado como alternativa SaaS. Técnicamente ofrece una experiencia completa de observabilidad, monitoreo de infraestructura, APM, logs y alertas. El problema es que su modelo de costo, dependencia de proveedor y alojamiento externo de datos no encajan con un laboratorio que exige Docker Compose y control local. Además, habría reducido la evidencia de aprovisionamiento propio que la rúbrica valora.

Zabbix 6.x fue seleccionado porque integra recolección, almacenamiento, interfaz web, triggers, actions, dashboards y API en una sola plataforma. Esto permitió configurar hosts, templates, items y media types con scripts, mantener un entorno autocontenido y demostrar alertas sin depender de servicios pagos. Su curva de configuración se compensó con la reproducibilidad del despliegue.

IV. DISEÑO DE LA SOLUCIÓN

La arquitectura se compone de una red Docker orquestada con Docker Compose. El núcleo de monitoreo incluye Zabbix Server 6.x personalizado, PostgreSQL como base de datos de Zabbix y el frontend web Nginx/PHP de Zabbix. La infraestructura monitoreada incluye web-service, db-service, dns-service y ftp-service, cada uno asociado a un agente Zabbix Agent 2.

El servicio web fue extendido para agregar valor operativo. Además del frontend HTML, expone endpoints de salud, resumen, telemetría, estado de base de datos, incidentes, analíticas, cumplimiento y métricas Prometheus. MariaDB persiste muestras de telemetría e incidentes para mostrar que el monitoreo no solo observa conectividad, sino también una aplicación con estado y escritura real.

Las alertas se diseñaron en dos niveles. MailHog captura correos de laboratorio y permite mostrar mensajes de problema y recuperación sin enviar spam a cuentas reales. Adicionalmente, se dejó documentado un canal SMTP real del dominio negociocontigo.com como camino de escalamiento hacia producción. El despliegue público se protegió con HTTPS mediante subdominios: web-zabbix, zabbix y mailhog-zabbix.

La publicación en una VPS permitió validar el proyecto fuera del equipo local. El profesor puede acceder por navegador a la consola, a Zabbix y a MailHog. Esto aporta evidencia práctica: no se trata de un Compose que solo corre

en el computador del estudiante, sino de una plataforma desplegada y accesible por Internet con certificados HTTPS.

Tabla II. Inventario funcional de servicios.

web-service: Frontend, API, telemetría y carga; monitoreo HTTP, /health, /metrics, API y agente.
db-service: MariaDB para datos de demo; monitoreo TCP 3306, estado API y agente.
dns-service: Resolución CoreDNS; monitoreo TCP/UDP 53 y agente.
ftp-service: Servicio VSFTPD; monitoreo de puerto 21, trigger y agente.
mailhog: SMTP de laboratorio; evidencia correos de problema y recuperación.
zabbix-server: Motor de monitoreo; items, triggers, problems, actions y API.

A. Modelo de monitoreo

El modelo de monitoreo combina tres enfoques. Primero, el agente entrega métricas del sistema como CPU, memoria, disco y disponibilidad del host. Segundo, los checks TCP/HTTP validan la respuesta funcional de servicios concretos. Tercero, el web escenario público revisa el recorrido HTTPS de la consola, de modo que una falla de frontend o gateway también se convierta en evento operativo.

Los triggers se definieron para detectar ausencia de datos, caída de agente y falta de respuesta de servicios. Esta separación es importante: una falla de agente indica pérdida de visibilidad del host, mientras que una falla de TCP o HTTP indica indisponibilidad funcional. Durante la demo se pueden mostrar ambos conceptos y explicar por qué monitorear solo el contenedor no basta.

El dashboard de Zabbix se complementa con la consola del proyecto. Zabbix conserva la historia oficial de items y problems; la consola entrega una vista didáctica para el evaluador, con botones de prueba, gráficas operativas, rutas más consultadas, cargas recientes, matriz de cumplimiento y estado de base de datos.

V. IMPLEMENTACIÓN

El proyecto quedó empaquetado en contenedores Docker. El archivo docker-compose.yml cubre el entorno local, mientras docker-compose.vps.yml adapta el despliegue público en la VPS. La imagen personalizada de Zabbix Server se construye desde docker/zabbix-server/Dockerfile y monta configuración adicional desde zabbix-config, cumpliendo el requerimiento de imagen propia y archivos de configuración como volúmenes.

El aprovisionamiento de Zabbix se automatizó con scripts Python. Se creó el grupo “Proyecto 7 - Infraestructura Docker”, se registraron los hosts web-host, db-host, dns-host y ftp-host, se asociaron templates Linux by Zabbix agent y se definieron checks específicos de servicio. También se configuró el web escenario público del portal y los media types para envío de alertas.

La consola web se diseñó para que la sustentación no dependa solo de capturas. Desde el navegador se puede verificar salud, crear incidentes, ejecutar cargas controladas y actualizar gráficas. El backend registra rutas observadas, respuestas exitosas y fallidas, cargas recientes, telemetría y estado de MariaDB. El endpoint /api/compliance traduce la rúbrica en una matriz verificable y /metrics expone indicadores compatibles con herramientas de monitoreo.

En producción se publicaron tres frentes: la consola del proyecto, Zabbix y MailHog protegido por autenticación. Esto permite al evaluador revisar el sistema en práctica desde Internet, con certificados HTTPS y sin acceder

directamente a la VPS. El README documenta inventario, URLs, credenciales de demostración, pasos de despliegue y comandos de prueba.

Para evitar dependencia de una versión inestable de Artillery, se fijó artillery@2.0.20 en los scripts y en la documentación. Durante la validación se observó que artillery@latest fallaba en la VPS con Node 22 por dependencias AWS ausentes. Fijar la versión mejora la reproducibilidad y reduce el riesgo durante la sustentación.

A. Endpoints de valor agregado

Tabla III. Endpoints principales de la consola.

/health: Verifica disponibilidad básica y permite check HTTP simple.
/api/live: Muestra estado operativo: requests, DB, SLO y cargas recientes.
/api/db/status: Valida persistencia MariaDB y evidencia backend con datos reales.
/api/charts: Alimenta gráficas del portal y facilita análisis histórico visual.
/api/compliance: Muestra matriz 13/13 y conecta la demo con la rúbrica.
/metrics: Exporter estilo Prometheus que amplía observabilidad del servicio web.

La API no se agregó como adorno. Cada endpoint tiene una función demostrable: /health sirve para disponibilidad, /api/live para estado agregado, /api/db/status para dependencia de datos, /api/compliance para evidenciar el cumplimiento del enunciado y /metrics para exponer indicadores que Zabbix puede consultar. Esto convierte el portal en un servicio con superficie de monitoreo propia.

MariaDB se usa para persistir muestras de telemetría e incidentes. Artillery y los botones de la consola generan escrituras que luego pueden verse desde /api/db/status y desde las gráficas. Esta decisión permite demostrar que Zabbix también puede monitorear dependencias internas de una aplicación, no solo puertos abiertos.

VI. PRUEBAS

La validación se organizó según las pruebas mínimas esperadas y se complementó con carga, auditoría y revisión de cumplimiento. Se ejecutaron pruebas de dashboard en tiempo real, caída controlada, envío de alertas, métricas históricas, Artillery smoke, Artillery live y consulta de endpoints públicos.

Tabla IV. Resultados principales de validación.

Dashboard y métricas: Cumple: Latest data, dashboard Zabbix y consola web con gráficas.
Caída controlada: Cumple: web, DB, DNS y FTP generaron problema y recuperación.
Alertas MailHog: Cumple: correos de Problem y Resolved visibles; 220 mensajes acumulados.
Métricas históricas: Cumple: Zabbix conserva series y la app registra 823 filas de telemetría.
Artillery live: Cumple: 2061 requests, 505 VUs, 0 fallidos, p95 45.2 ms.
Artillery smoke: Cumple: 96 requests, 8 VUs, 0 fallidos, p95 22.9 ms.
Matriz de cumplimiento: Cumple: 13/13 criterios, 100%.

A. Prueba de dashboard en tiempo real

El dashboard de Zabbix permitió visualizar hosts, disponibilidad y métricas recolectadas por los agentes. En paralelo, la consola web mostró gráficas de solicitudes, telemetría y SLO. Esta doble vista fue útil para explicar la diferencia entre una herramienta de monitoreo centralizada y una pantalla de demostración orientada a la sustentación.

Durante la prueba se consultaron endpoints públicos y se generaron muestras de telemetría. La actualización dinámica se evidenció en el incremento de rutas observadas, cargas recientes y filas persistidas en MariaDB. Esto cumple la expectativa de visualizar métricas en tiempo real y además muestra que el backend tiene comportamiento medible.

B. Simulación de caída de host y servicios

Para la simulación de fallas se detuvieron los contenedores de forma secuencial y se esperó el ciclo de sondeo de Zabbix. La plataforma detectó fallas como “HTTP web-service no responde”, “MySQL db-service no responde”, “DNS dns-service TCP no responde” y “FTP ftp-service no responde”. Posteriormente, al reiniciar los contenedores, Zabbix registró eventos de recuperación y MailHog recibió los correos correspondientes.

La prueba demostró que el sistema no solo identifica la caída del host, sino también la indisponibilidad de servicios concretos. Esto es relevante porque un contenedor podría estar activo y aun así fallar en su puerto funcional. Al separar agent.ping, checks TCP y web escenarios se obtiene una lectura más precisa de la causa de la falla.

C. Pruebas de carga con Artillery

La prueba de carga live se ejecutó con Artillery contra la consola pública. El escenario incluyó navegación de frontend, consultas a /api/live, /api/charts, /api/compliance, /metrics y /api/slo, además de escritura de telemetría e incidentes. El sistema completó 505 usuarios virtuales sin fallos y mantuvo una latencia p95 de 45.2 ms, valor suficiente para una demostración académica con tráfico controlado.

El smoke test ejecutó 96 solicitudes con 8 usuarios virtuales, sin usuarios fallidos y con p95 de 22.9 ms. Esta prueba rápida sirve antes de la sustentación para confirmar que el portal, la API, la base de datos y las rutas de carga siguen funcionando. El escenario live, en cambio, se reserva para mostrar tráfico visible durante la exposición.

Después de las pruebas, el estado final quedó sin problemas activos en Zabbix. La consulta por API reportó 0 problemas activos, y los servicios web, DB, DNS, FTP, Zabbix Server, Zabbix Web, MailHog y MailHog gate permanecieron arriba. La consola pública mostró SLO de laboratorio en estado cumple y la matriz /api/compliance volvió a 100%.

VII. DISCUSIÓN DE RESULTADOS

Los resultados confirman que la solución cubre el flujo operativo completo. Los agentes aportan métricas de recursos, los checks de servicio validan disponibilidad funcional y los triggers transforman fallas en eventos accionables. La existencia de correos de problema y recuperación demuestra que el monitoreo no queda encerrado en el dashboard, sino que puede integrarse a canales de notificación.

El uso de una aplicación real fortalece la sustentación. Al tener backend, base de datos, telemetría e incidentes, es posible explicar escenarios de degradación y no solo caídas binarias. Artillery genera tráfico observable, mientras la consola permite ver requests, rutas, cargas recientes y SLO. Esto ayuda a discutir diferencias entre disponibilidad, rendimiento, saturación y recuperación.

La principal limitación es que el SLO de la consola se calcula sobre contadores de proceso y se reinicia si el contenedor web se recrea. Zabbix, por su parte, conserva el histórico real de métricas y eventos. En un ambiente productivo, el siguiente paso sería persistir el cálculo de SLO en base de datos o exportarlo a una herramienta de series de tiempo, además de endurecer seguridad, usuarios y backups.

Otra consideración es la seguridad del despliegue. Zabbix y MailHog están publicados para facilitar la evaluación, pero en un entorno real deberían limitarse por VPN, lista de IPs, autenticación robusta y políticas de mínimo privilegio. El proyecto separa credenciales de demostración de credenciales root y no distribuye llaves SSH en los entregables del grupo.

La elección de MailHog es adecuada para laboratorio porque permite inspeccionar correos sin enviar mensajes reales durante cada prueba. Para producción se documentó un canal SMTP del dominio, pero se mantiene como escalamiento y no como reemplazo de MailHog. Esto evita ruido en cuentas reales y conserva una evidencia clara para el evaluador.

VIII. RELACIÓN CON LA RÚBRICA

Tabla V. Trazabilidad contra requerimientos y evaluación.

Diseño de solución: Cumple: arquitectura Zabbix + Docker + servicios + alertas.

Validación y selección: Cumple: comparación de alternativas y pruebas reproducibles.

Construcción de prototipo: Cumple: VPS pública, HTTPS, backend, DB, SLO y Artillery.

Requerimientos Docker: Cumple: Compose, imagen Zabbix personalizada y volúmenes.

Pruebas mínimas: Cumple: dashboard, caída controlada, MailHog e históricos.

Valor agregado: Cumple: consola web, /metrics, analíticas y matriz 13/13.

La rúbrica otorga mayor peso al diseño de soluciones, la estructuración del proceso de validación y la construcción del prototipo. El proyecto responde a esos tres frentes: presenta una arquitectura concreta, justifica Zabbix frente a alternativas, ejecuta pruebas medibles y entrega un prototipo público funcional.

La matriz /api/compliance refuerza esta trazabilidad porque convierte los requerimientos del enunciado en una verificación visible. El resultado final validado fue 13 de 13 criterios cumplidos, con 100% de cumplimiento. Este indicador no sustituye la revisión del profesor, pero ayuda a ordenar la sustentación y a mostrar que los entregables cubren infraestructura, Zabbix, monitoreo avanzado, Docker y documentación.

IX. ASPECTOS OPERATIVOS Y LECCIONES TÉCNICAS

Una decisión importante fue separar el monitoreo de infraestructura del comportamiento de la aplicación. Zabbix observa hosts, agentes, puertos y escenarios web; la consola observa rutas, carga, telemetría, incidentes y SLO. Esta separación evita mezclar responsabilidades y permite explicar dos niveles de visibilidad: infraestructura base y experiencia funcional del servicio.

El aprovisionamiento por API redujo errores de configuración manual. En lugar de crear hosts y triggers desde la interfaz uno por uno, el script de provisión deja un proceso repetible. Si el entorno se reconstruye, el equipo puede volver a generar grupos, templates, media types y acciones. Esta característica es valiosa para la rúbrica porque demuestra proceso, no solo resultado final.

El despliegue en la VPS también dejó una lección práctica: publicar servicios de monitoreo exige balancear facilidad de evaluación y seguridad. Para la sustentación se publicaron subdominios HTTPS y se protegió MailHog con autenticación. Sin embargo, en producción se debería restringir Zabbix y MailHog por VPN o lista de IPs, rotar

credenciales, separar usuarios por rol y evitar exponer interfaces administrativas a Internet.

Otra lección surgió durante las pruebas de carga. Usar artillery@latest produjo un fallo por dependencias en Node 22, por lo que se fijó artillery@2.0.20. Esta decisión mejora la reproducibilidad y evita que una actualización externa rompa la demo. En proyectos de infraestructura, fijar versiones es tan importante como escribir la configuración, porque una herramienta no determinística puede invalidar la evidencia el día de la sustentación.

El proyecto también evidencia la diferencia entre monitoreo y observabilidad. Monitorear responde si algo está arriba o abajo; observar permite explicar por qué ocurrió, qué rutas se afectaron, cuántas solicitudes pasaron, si hubo escritura en base de datos y si el SLO se mantuvo. La combinación de Zabbix, /metrics, Artillery y la consola web permite contar esa historia de forma completa.

Tabla VI. Riesgos operativos y mitigaciones aplicadas.

Caída de servicio monitoreado: Triggers por HTTP, TCP y recuperación controlada del contenedor durante la demo.

Fallo de base de datos: Check TCP 3306 y endpoint /api/db/status monitoreado.

Alertas no enviadas: MailHog para laboratorio y canal SMTP documentado para producción.

Herramienta de carga inestable: Versión Artillery fijada en 2.0.20 y smoke test corto.

Exposición pública excesiva: HTTPS, autenticación en MailHog y recomendación de VPN o lista de IPs.

La evaluación final debe interpretarse en conjunto: los resultados de Artillery muestran rendimiento bajo carga controlada, Zabbix evidencia la detección de problemas y MailHog confirma el canal de notificación. Ninguna de estas evidencias por separado demuestra toda la solución; juntas muestran el ciclo operativo completo de monitoreo, falla, alerta, recuperación y análisis.

El informe, la presentación y el guion se alinearon con ese ciclo. Los primeros minutos de la sustentación explican problema y arquitectura; la parte central muestra configuración Zabbix; el cierre técnico ejecuta carga y falla controlada. Esta distribución permite que cada integrante participe sin perder el hilo principal del proyecto.

X. CONCLUSIONES

Se implementó una plataforma funcional de monitoreo de infraestructura con Zabbix 6.x, Docker Compose, PostgreSQL, MailHog y cuatro servicios monitoreados: web, base de datos, DNS y FTP. La solución cumple los requerimientos del enunciado: servidor Zabbix, frontend, agentes, hosts, grupos, templates, triggers, dashboard, alertas y README con inventario y URLs.

La propuesta supera el monitoreo básico al incorporar una aplicación web real con backend Node.js, MariaDB, endpoints JSON, exporter /metrics, SLO, gráficas, incidentes, pruebas de carga y matriz de cumplimiento. Esta ampliación permite sustentar el proyecto como una plataforma de observabilidad aplicada y no únicamente como una instalación de laboratorio.

Las pruebas demostraron detección de fallas, envío de alertas, recuperación, métricas históricas y comportamiento bajo carga. La publicación en VPS con HTTPS facilita que el evaluador revise la solución en vivo y que el equipo presente evidencias reproducibles dentro del tiempo máximo de sustentación.

Como trabajo futuro se propone persistir el SLO en una base de datos de series temporales, agregar dashboards comparativos por servicio, restringir la exposición pública de Zabbix y MailHog, automatizar backups y conectar alertas reales a un canal de incidentes como correo institucional, Telegram o Slack.

REFERENCIAS

- [1] Zabbix LLC, “Zabbix Documentation 6.0,” 2026. [En línea]. Disponible: <https://www.zabbix.com/documentation/6.0/>
- [2] Docker Inc., “Docker Compose overview,” 2026. [En línea]. Disponible: <https://docs.docker.com/compose/>
- [3] MailHog, “Web and API based SMTP testing,” GitHub, 2026. [En línea]. Disponible: <https://github.com/mailhog/MailHog>
- [4] Artillery, “Load testing documentation,” 2026. [En línea]. Disponible: <https://www.artillery.io/docs>
- [5] PostgreSQL Global Development Group, “PostgreSQL Documentation,” 2026. [En línea]. Disponible: <https://www.postgresql.org/docs/>
- [6] Caddy, “Automatic HTTPS,” 2026. [En línea]. Disponible: <https://caddyserver.com/docs/automatic-https>
- [7] MariaDB Foundation, “MariaDB Server Documentation,” 2026. [En línea]. Disponible: <https://mariadb.org/documentation/>
- [8] CoreDNS, “CoreDNS Manual,” 2026. [En línea]. Disponible: <https://coredns.io/manual/toc/>